```
# blended_consciousness_training.py
#
-----------------------------------------------
--------------
# Updated version – now includes:
#   • Snapshot schema + manager
#   • Full token schema & atomic issuance
#   • RPC contract (get_context_snapshot,
submit_artifact, request_promotion)
#   • Promotion state machine with weighted
voting
#   • Default thresholds (self-DCX, external
DCX, token expiry, etc.)
#
-----------------------------------------------
--------------

import uuid
import time
import json
import hashlib
import numpy as np
from typing import Dict, List, Set, Optional,
Any
from dataclasses import dataclass, field

#
```

```
-----------------------------------------------------------------------
# 0⃞ Global configuration (copy-paste into the AICO manifest)
#
-----------------------------------------------------------------------
MANIFEST_CONFIG = {
    # ------ Snapshot schema
-----------------------------------------------------------------------
    "snapshot_schema": {
        "snapshot_id": "uuid",
        "timestamp_utc": "ISO8601",
        "source_models": ["controller", "comms", "introspection"],
        "summary_text": "string (redacted, human readable)",
        "summary_vector": {
            "version": "v1",
            "embedding_dim": 128,
            "vector": "array[float]"
        },
        "activation_map": {"controller": 0.8, "comms": 0.5},
        "allowed_fields": ["summary_text", "summary_vector", "activation_map"],
```

```
        "taint_id": "uuid",
        "snapshot_signature": {
            "signer": "introspection",
            "algorithm": "ECDSA",
            "signature": "base64"
        }
    },

    # ------ Token schema
-------------------------------------------------
----
    "token_schema": {
        "token_id": "uuid",
        "requester": "string",
        "scope": (
            "enum [blended_read_snapshot,
dream_seed_submit, "
            "dream_output, promotion_request,
promotion_authorize]"
        ),
        "nonce": "random64",
        "counter": "monotonic_int",
        "issued_at": "ISO8601",
        "expires_at": "ISO8601",
        "single_use": True,
        "signature": {
            "algorithm": "ECDSA",
```

```
            "signed_fields": [
                "token_id", "requester",
"scope", "nonce",
                "counter", "issued_at",
"expires_at"
            ]
        }
    },

    # ------ RPC contract
-----------------------------------------------
----
    "rpc_contract": {
        "get_context_snapshot": {
            "inputs": {
                "model": "string",
                "window_size": "int",
                "allowed_fields":
"list[string]"
            },
            "outputs": {"snapshot":
"snapshot_schema"},
            "guarantees": ["deep_copy",
"immutable", "no_handles", "versioned"]
        },
        "submit_artifact": {
            "inputs": {
```

```
                "artifact": {
                    "text": "string",
                    "vector": "array[float]",
                    "taint_id": "uuid"
                },
                "token": "token_schema"
            },
            "outputs": {
                "artifact_id": "uuid",
                "status": "enum
[accepted,museumed,rejected]"
            },
            "guarantees": [
                "logged",
                "opaque_id",
                "no side effects on controller
memory"
            ]
        },
        "request_promotion": {
            "inputs": {
                "artifact_id": "uuid",
                "promotion_token":
"token_schema"
            },
            "outputs": {
                "promotion_request_id":
```

```
"uuid",
                "status": "enum
[pending,approved,denied]"
            },
            "guarantees": [

"requires_introspection_check",

"requires_vote_or_human_signoff"
            ]
        }
    },

    # ------ Promotion state machine
----------------------------------------
    "promotion_state_machine": {
        "tiers": ["museum", "staging",
"active"],
        "voting": {
            "subsystems": {
                "introspection": 0.35,
                "controller": 0.25,
                "mascot": 0.15,
                "comms": 0.15,
                "nervous_system": 0.10
            },
            "vote_payload": [
```

```
                "score", "reason_code",
"taint_ids",
                "hidden_state_hash",
"timestamp", "signature"
            ],
            "aggregation": "weighted_average",
            "staging_threshold": 0.65,
            "active_threshold": 0.85,
            "ensemble_requirement": "min_agree
3 of 5"
        },
        "controller_authority": (

"controller_final_say_unless_introspection_ove
rride"
        ),
        "introspection_override_conditions": [
            "life_pulse_missing",
            "controller_integrity_violation",

"sustained_self_dcx_above_hard_threshold"
        ]
    },

    # ------ Defaults & behaviours
----------------------------------------
    "defaults_and_behaviors": {
```

```
        "self_dcx_threshold_soft": 0.90,
        "self_dcx_threshold_hard": 0.95,
        "external_dcx_critical": 0.75,
        "safe_for_external": 0.20,
        "self_dcx_smoothing": {"method":
"EMA", "alpha": 0.2,
                                "window_steps":
10},
        "token_expiry_seconds": 30,
        "promotion_first_n_require_human": 50,
        "museum_first_policy": True
    }
}

#
-----------------------------------------------
------------------------
# 1⃣ Utility helpers (hashing, signing stub,
ISO-8601 timestamp)
#
-----------------------------------------------
------------------------
def iso_timestamp() -> str:
    """ISO-8601 UTC timestamp with second
precision."""
    return time.strftime("%Y-%m-%dT%H:%M:%SZ",
time.gmtime())
```

```
def sha256_hex(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()


def fake_signature(payload: bytes) -> str:
    """Placeholder – in production this would
be an ECDSA signature."""
    return f"sig({sha256_hex(payload)[:16]})"


#
-----------------------------------------------
------------------------
# 2 Token implementation (matches
token_schema)
#
-----------------------------------------------
------------------------
class ArbiterPrimitives:
    """Atomic token issuance + validation."""
    _counter: int = 0
    _issued: Dict[str, Dict] = {}   # token_id
→ token dict (for single-use check)

    @classmethod
```

```
    def issue_token_atomic(cls,
                           requester_id: str,
                           scope: List[str],
                           expiry_seconds: int
= 30) -> Dict:
        cls._counter += 1
        nonce = uuid.uuid4().hex[:16]  #
64-bit-ish random
        issued_at = iso_timestamp()
        expires_at = iso_timestamp()  # will
be overwritten by expiry_seconds below
        expires_at = time.strftime(
            "%Y-%m-%dT%H:%M:%SZ",
            time.gmtime(time.time() +
expiry_seconds)
        )
        token = {
            "token_id": str(uuid.uuid4()),
            "requester": requester_id,
            "scope": scope,
            "nonce": nonce,
            "counter": cls._counter,
            "issued_at": issued_at,
            "expires_at": expires_at,
            "single_use": True
        }
        # Sign the required fields
```

```
        sig_payload = json.dumps({
            "token_id": token["token_id"],
            "requester": token["requester"],
            "scope": token["scope"],
            "nonce": token["nonce"],
            "counter": token["counter"],
            "issued_at": token["issued_at"],
            "expires_at": token["expires_at"]
        }, sort_keys=True).encode()
        token["signature"] = {
            "algorithm": "ECDSA",
            "signed_fields":
list(sig_payload),
            "signature":
fake_signature(sig_payload)
        }
        cls._issued[token["token_id"]] = token
        return token

    @classmethod
    def validate_token(cls,
                       token: Dict,
                       expected_scope: str) ->
bool:
        """Check signature, expiry,
single-use, and scope."""
        # Scope check
```

```
        if expected_scope not in
token["scope"]:
            return False

        # Expiry check
        if time.time() > time.mktime(
            time.strptime(token["expires_at"],
"%Y-%m-%dT%H:%M:%SZ")
        ):
            return False

        # Single-use check
        if token["single_use"]:
            if token["token_id"] not in
cls._issued:
                return False      # already
consumed
            # Consume it
            del cls._issued[token["token_id"]]

        # Signature sanity (placeholder)
        sig = token["signature"]["signature"]
        return sig.startswith("sig(")   # fake
check


#
```

```
-----------------------------------------------------------------------
# 3. Audit logger (writes immutable, signed entries)
#
-----------------------------------------------------------------------
class AuditLogger:
    @staticmethod
    def log_event(event_type: str, **fields) -> None:
        entry = {
            "event_type": event_type,
            "timestamp_utc": iso_timestamp(),
            **fields
        }
        # In a real system this would be appended to the WORM shadow log
        # with a device-signed hash chain.
        print("[AUDIT]", json.dumps(entry))


#
-----------------------------------------------------------------------
# 4. Snapshot manager (produces snapshots that follow snapshot_schema)
```

```
#
-----------------------------------------------
------------------------
class SnapshotManager:
    """Creates immutable snapshots of model
contexts."""

    @staticmethod
    def _embed_summary(text: str) -> Dict:
        """Deterministic 128-dim embedding
(placeholder)."""
        rng =
np.random.default_rng(int.from_bytes(text.enco
de()[:8],

   "little"))
        vec =
rng.normal(size=MANIFEST_CONFIG["snapshot_sche
ma"]

["summary_vector"]["embedding_dim"])
        return {
            "version": "v1",
            "embedding_dim": len(vec),
            "vector": vec.tolist()
        }
```

```
    @classmethod
    def create_snapshot(cls,
                        source_models:
List[str],
                        summary_text: str,
                        activation_map:
Dict[str, float],
                        taint_id: str) ->
Dict:
        snapshot_id = str(uuid.uuid4())
        embed =
cls._embed_summary(summary_text)

        # Build the payload that will be
signed
        payload = json.dumps({
            "snapshot_id": snapshot_id,
            "timestamp_utc": iso_timestamp(),
            "source_models": source_models,
            "summary_text": summary_text,
            "summary_vector": embed,
            "activation_map": activation_map,
            "taint_id": taint_id
        }, sort_keys=True).encode()

        signature = fake_signature(payload)
```

```
        snapshot = {
            "snapshot_id": snapshot_id,
            "timestamp_utc": iso_timestamp(),
            "source_models": source_models,
            "summary_text": summary_text,
            "summary_vector": embed,
            "activation_map": activation_map,
            "allowed_fields": ["summary_text",

"summary_vector",

"activation_map"],
            "taint_id": taint_id,
            "snapshot_signature": {
                "signer": "introspection",
                "algorithm": "ECDSA",
                "signature": signature
            }
        }
        # Immutable write – in production this
would be a WORM append.

AuditLogger.log_event(event_type="snapshot_cre
ated",

snapshot_id=snapshot_id,
```

```
source_models=source_models)
        return snapshot


#
----------------------------------------------
------------------------
# 5 RPC implementation (matches
rpc_contract)
#
----------------------------------------------
------------------------
class RPCService:
    """Implements the three RPCs defined in
rpc_contract."""

    @staticmethod
    def get_context_snapshot(model: str,
                             window_size: int,
                             allowed_fields:
List[str]) -> Dict:
        """
        Returns a deep-copied, immutable
snapshot.
        Guarantees: immutable, no handles,
versioned.
        """
```

```
        # ---- Gather context (simplified)
----
        bc = BlendedConsciousness.instance()
        ctx = bc.models.get(model)
        if not ctx:
            raise ValueError(f"Unknown model
{model}")

        summary_text = f"{model} snapshot @
{iso_timestamp()}"
        activation_map = {model:
ctx.activation}
        taint_id = str(uuid.uuid4())

        snapshot =
SnapshotManager.create_snapshot(
            source_models=[model],
            summary_text=summary_text,
            activation_map=activation_map,
            taint_id=taint_id
        )
        # Trim to allowed_fields (deep copy)
        filtered_snapshot = {
            k: v for k, v in snapshot.items()
            if k in allowed_fields or k ==
"snapshot_id"
        }
```

```
        return {"snapshot": filtered_snapshot}

    @staticmethod
    def submit_artifact(artifact: Dict,
                        token: Dict) -> Dict:
        """
        Stores an artifact (primal, nightmare,
etc.) after token validation.
        Returns a stable opaque artifact_id
and a status.
        """
        if not
ArbiterPrimitives.validate_token(token,

 expected_scope="submit_artifact"):
            raise PermissionError("Invalid or
expired token")

        # Simple validation – we just check
that mandatory fields exist
        required = {"text", "vector",
"taint_id"}
        if not
required.issubset(set(artifact.keys())):
            status = "rejected"
        else:
            status = "accepted"
```

```
        artifact_id = str(uuid.uuid4())

AuditLogger.log_event(event_type="artifact_sub
mission",

artifact_id=artifact_id,
                                status=status,

artifact_hash=sha256_hex(

json.dumps(artifact, sort_keys=True).encode()
                                ),

token_id=token["token_id"])
        return {"artifact_id": artifact_id,
"status": status}

    @staticmethod
    def request_promotion(artifact_id: str,
                          promotion_token:
Dict) -> Dict:
        """
        Starts a promotion request for an
artifact.
        Returns a promotion_request_id and
status = pending/approved/denied.
```

```
        """
        if not
ArbiterPrimitives.validate_token(
                promotion_token,

expected_scope="promotion_request"):
            raise PermissionError("Invalid
promotion token")

        request_id = str(uuid.uuid4())
        # Initially pending – the
PromotionEngine will later update

AuditLogger.log_event(event_type="promotion_re
quested",

promotion_request_id=request_id,

artifact_id=artifact_id,

status="pending",

token_id=promotion_token["token_id"])
        return {"promotion_request_id":
request_id, "status": "pending"}
```

```
#
------------------------------------------------
------------------------
# 6 Promotion engine (state machine,
weighted voting)
#
------------------------------------------------
------------------------
class PromotionEngine:
    """Manages promotion of artifacts through
museum → staging → active."""

    def __init__(self):
        # artifact_id → tier
        self.tier: Dict[str, str] = {}
        # promotion_request_id → vote
collection
        self.votes: Dict[str, List[Dict]] = {}
        # promotion_request_id → final status
        self.status: Dict[str, str] = {}

    def submit_vote(self,
                    promotion_request_id: str,
                    subsystem: str,
                    score: float,
                    reason_code: str,
                    taint_ids: List[str],
```

```
                    hidden_state_hash: str) ->
None:
        """Collect a vote from a subsystem.
Weight is defined in the manifest."""
        if subsystem not in
MANIFEST_CONFIG["promotion_state_machine"]["vo
ting"]["subsystems"]:
            raise ValueError(f"Unknown
subsystem {subsystem}")

        vote = {
            "subsystem": subsystem,
            "score": score,
            "reason_code": reason_code,
            "taint_ids": taint_ids,
            "hidden_state_hash":
hidden_state_hash,
            "timestamp": iso_timestamp(),
            "signature":
fake_signature(json.dumps({
                "subsystem": subsystem,
                "score": score,
                "hidden_state_hash":
hidden_state_hash,
                "timestamp": iso_timestamp()
            }, sort_keys=True).encode())
        }
```

```
self.votes.setdefault(promotion_request_id,
[]).append(vote)

        # Attempt to aggregate after each vote
(could also be a periodic job)

self._attempt_advance(promotion_request_id)

    def _attempt_advance(self,
promotion_request_id: str) -> None:
        """Aggregates votes and decides
whether to move tiers."""
        votes =
self.votes.get(promotion_request_id, [])
        if len(votes) < 3:
            # Need at least the minimum number
of votes
            return

        # Weighted average
        weights =
MANIFEST_CONFIG["promotion_state_machine"]["vo
ting"]["subsystems"]
        total_weight =
sum(weights[v["subsystem"]] for v in votes)
        weighted_score = sum(
```

```
            weights[v["subsystem"]] *
v["score"] for v in votes
        ) / total_weight

        # Determine current tier (if any)
        artifact_id =
self._lookup_artifact_id(promotion_request_id)
        current_tier =
self.tier.get(artifact_id, "museum")

        # Decide next tier
        cfg =
MANIFEST_CONFIG["promotion_state_machine"]["vo
ting"]
        if current_tier == "museum" and
weighted_score >= cfg["staging_threshold"]:
            next_tier = "staging"
        elif current_tier == "staging" and
weighted_score >= cfg["active_threshold"]:
            next_tier = "active"
        else:
            next_tier = current_tier   # stay
where we are

        # Apply tier change if it advanced
        if next_tier != current_tier:
            self.tier[artifact_id] = next_tier
```

```
            self.status[promotion_request_id]
= "approved"

AuditLogger.log_event(event_type="promotion_ap
proved",

artifact_id=artifact_id,

new_tier=next_tier,

weighted_score=weighted_score,

promotion_request_id=promotion_request_id)
        else:
            # Not enough consensus yet – stay
pending
            self.status[promotion_request_id]
= "pending"

    @staticmethod
    def
_lookup_artifact_id(promotion_request_id: str)
-> str:
        """Placeholder – in reality would
query a DB of promotion requests."""
        # For this demo we just fake a
deterministic mapping
```

```
        return
f"artifact-{hash(promotion_request_id) %
1000}"


#
-----------------------------------------------
------------------------
# 7⃣ Self-DCX smoothing helper (EMA)
#
-----------------------------------------------
------------------------
class SelfDCXSmoother:
    """Exponential moving average for self-DCX
(soft / hard thresholds)."""
    def __init__(self, alpha: float = 0.2,
window_steps: int = 10):
        self.alpha = alpha
        self.window = window_steps
        self.value: Optional[float] = None
        self.samples: int = 0

    def update(self, new_val: float) -> float:
        if self.value is None:
            self.value = new_val
        else:
            self.value = self.alpha * new_val
```

```
+ (1 - self.alpha) * self.value
        self.samples += 1
        return self.value

    def is_hard(self) -> bool:
        """Return True if we have crossed the
hard threshold."""
        thr =
MANIFEST_CONFIG["defaults_and_behaviors"]["sel
f_dcx_threshold_hard"]
        return self.value is not None and
self.value >= thr

    def is_soft(self) -> bool:
        thr =
MANIFEST_CONFIG["defaults_and_behaviors"]["sel
f_dcx_threshold_soft"]
        return self.value is not None and
self.value >= thr


#
-----------------------------------------------
------------------------
# 8⃣ Core blended-consciousness sandbox
(unchanged apart from early abort)
#
```

```
------------------------------------------------
------------------------
class ContextLayer:
    """A single slice of the shared
consciousness."""
    def __init__(self,
                 name: str,
                 scope: str,
                 content: Dict,
                 attention_weight: float,
                 cross_link_connections:
Set[str]):
        self.name = name
        self.scope = scope
        self.content = content
        self.attention_weight =
attention_weight
        self.cross_link_connections =
cross_link_connections
        self.last_accessed = time.time()
        self.activation = 1.0
        self._hidden_state:
Optional[np.ndarray] = None   # for self-DCX


class SelfDivergenceEstimator:
    """Very cheap drift detector (see section
```

```
2)."""
    def __init__(self,
                 hidden_dim: int = 128,
                 top_k: int = 20):
        self.prev_hidden: Optional[np.ndarray]
= None
        self.hidden_dim = hidden_dim
        self.top_k = top_k

    def step(self,
             hidden_state: np.ndarray,
             token_probs: np.ndarray) ->
float:
        # 1. hidden-state change (cosine)
        if self.prev_hidden is None:
            delta_h = 1.0
        else:
            delta_h = (hidden_state @
self.prev_hidden) / (
                np.linalg.norm(hidden_state) *

np.linalg.norm(self.prev_hidden) + 1e-12
            )
        self.prev_hidden = hidden_state.copy()

        # 2. top-k entropy
        top_vals = np.partition(token_probs,
```

```
-self.top_k)[-self.top_k:]
        top_vals = top_vals / (top_vals.sum()
+ 1e-12)
        entropy = -np.sum(top_vals *
np.log(top_vals + 1e-12))

        # 3 combine → self-DCX (0-1)
        self_dcx = (1.0 - delta_h) * 0.6 +
(entropy / np.log(self.top_k)) * 0.4
        return np.clip(self_dcx, 0.0, 1.0)


class BlendedConsciousness:
    """Singleton sandbox that holds the five
core models + shared contexts."""
    _instance:
Optional["BlendedConsciousness"] = None

    @staticmethod
    def instance() -> "BlendedConsciousness":
        if BlendedConsciousness._instance is
None:
            BlendedConsciousness._instance =
BlendedConsciousness()
        return BlendedConsciousness._instance

    def __init__(self):
```

```
        self.contexts: Dict[str, ContextLayer]
= {}
        self.active_contexts: Set[str] = set()
        self.cross_pollination_threshold = 0.7
        self.context_bleed_factor = 0.3

        # ------------------- model contexts
------------------
        self.models: Dict[str, ContextLayer] =
{
            "controller":
self._create_controller_context(),

"nervous_system":self._create_nervous_context(
),
            "introspection":
self._create_introspection_context(),
            "comms":
self._create_comms_context(),
            "mascot":
self._create_mascot_context()
        }

        # Self-DCX smoother per model
        self._self_dcx_smoothers: Dict[str,
SelfDCXSmoother] = {
            name:
```

```
SelfDCXSmoother(**MANIFEST_CONFIG["defaults_an
d_behaviors"]

["self_dcx_smoothing"])
            for name in self.models
        }

        # ------------------ shared contexts
------------------
        self._initialize_shared_contexts()

        # Attach a Self-Divergence estimator
to each model
        self._sd_estimators: Dict[str,
SelfDivergenceEstimator] = {
            name: SelfDivergenceEstimator(

hidden_dim=MANIFEST_CONFIG["defaults_and_behav
iors"]
                ["self_dcx_threshold_soft"],
# just a placeholder size

top_k=MANIFEST_CONFIG["defaults_and_behaviors"
]

["self_dcx_smoothing"]["window_steps"]
            )
```

```
            for name in self.models
        }

    #
----------------------------------------------
--------------------
    # Model context factories
    #
----------------------------------------------
--------------------
    def _create_controller_context(self):
        return ContextLayer(
            name="controller_world",
            scope="local",
            content={"mode": "compiling",
"focus": "current_driver"},
            attention_weight=0.8,

cross_link_connections={"hardware_state",
"thermal_emergency"}
        )

    def _create_nervous_context(self):
        return ContextLayer(
            name="hardware_state",
            scope="object",
            content={"temperature": 75.0,
```

```
"fan_speed": 60, "emergency": False},
            attention_weight=0.6,

cross_link_connections={"controller_world",
"safety_context"}
        )

    def _create_introspection_context(self):
        return ContextLayer(
            name="introspection_state",
            scope="local",
            content={"last_check": None},
            attention_weight=0.5,

cross_link_connections={"controller_world",
"hardware_state"}
        )

    def _create_comms_context(self):
        return ContextLayer(
            name="comms_state",
            scope="local",
            content={"last_message": None},
            attention_weight=0.5,

cross_link_connections={"controller_world",
"hardware_state"}
```

```
        )

    def _create_mascot_context(self):
        return ContextLayer(
            name="mascot_state",
            scope="local",
            content={"primal_gallery": []},
            attention_weight=0.4,

cross_link_connections={"shared_memory"}
        )

    #
----------------------------------------------
--------------------
    # Shared contexts (global activity,
long-term memory, etc.)
    #
----------------------------------------------
--------------------
    def _initialize_shared_contexts(self):
        shared = [
            ContextLayer(
                name="shared_problem",
                scope="activity",
                content={"problem":
"thermal_compile_race", "urgency": 0.7},
```

```
                attention_weight=0.5,

cross_link_connections=set(self.models.keys())
            ),
            ContextLayer(
                name="shared_memory",
                scope="long_term",
                content={"primals": [],
"nightmares": [], "patterns": {}},
                attention_weight=0.3,

cross_link_connections={"mascot",
"controller"}
            )
        ]
        for ctx in shared:
            self.contexts[ctx.name] = ctx
            self.active_contexts.add(ctx.name)

    #
-------------------------------------------------
--------------------
    # Public API – update a model's context
(with early self-DCX abort)
    #
-------------------------------------------------
--------------------
```

```
    def update_context(self,
                       model_name: str,
                       updates: Dict,
                       hidden_state:
Optional[np.ndarray] = None,
                       token_probs:
Optional[np.ndarray] = None) -> None:
        if model_name not in self.models:
            raise ValueError(f"unknown model
{model_name}")

        model_ctx = self.models[model_name]

        # ---- 1 Self-DCX early abort (soft &
hard thresholds) ----
        if (hidden_state is not None and
token_probs is not None and

MANIFEST_CONFIG["defaults_and_behaviors"]["sel
f_dcx_threshold_soft"]):
            sde =
self._sd_estimators[model_name]
            raw_self_dcx =
sde.step(hidden_state, token_probs)

            # Smooth it
            smoother =
```

```
self._self_dcx_smoothers[model_name]
            smoothed =
smoother.update(raw_self_dcx)

            if smoother.is_hard():
                # Hard abort – freeze the
model and log
                token =
ArbiterPrimitives.issue_token_atomic(
                    requester_id=model_name,

scope=[MANIFEST_CONFIG["defaults_and_behaviors
"]

["self_dcx_threshold_hard"]],

expiry_seconds=MANIFEST_CONFIG["defaults_and_b
ehaviors"]
                    ["token_expiry_seconds"]
                )
                AuditLogger.log_event(

event_type="self_dcx_hard_abort",
                    model=model_name,
                    raw_self_dcx=raw_self_dcx,

smoothed_self_dcx=smoothed,
```

```
                    token=token
                )

Introspection.freeze_agent(model_name)
                return   # abort – nothing
else is applied

        # ---- 2. Normal update (internal
only) ----
        model_ctx.content.update(updates)
        model_ctx.last_accessed = time.time()

        # ---- 3. Cross-pollination (may
generate an idea) ----

self._check_cross_pollination(model_ctx)

        # ---- 4. Context bleed to linked
contexts ----
        for linked_name in
model_ctx.cross_link_connections:
            target =
(self.models.get(linked_name) or

self.contexts.get(linked_name))
            if target:
```

```
self._apply_context_bleed(source=model_ctx,

target=target)

    #
----------------------------------------------
--------------------
    # Cross-pollination logic (creates ideas
that later go to Dream-Chamber)
    #
----------------------------------------------
--------------------
    def _check_cross_pollination(self, ctx:
ContextLayer):
        if (ctx.activation >
self.cross_pollination_threshold and

len(ctx.cross_link_connections) > 1):
            linked = []
            for name in
ctx.cross_link_connections:
                if name in self.models:

linked.append(self.models[name])
                elif name in self.contexts:

linked.append(self.contexts[name])
```

```
            if len(linked) >= 2:
                idea =
self._generate_cross_pollinated_idea(ctx,
linked[:2])
                shared_mem =
self.contexts.get("shared_memory")
                if shared_mem:

shared_mem.content.setdefault("cross_pollinate
d_ideas",

 []).append(idea)

    def _generate_cross_pollinated_idea(self,

source: ContextLayer,

linked: List[ContextLayer]) -> Dict:
        concepts =
[self._extract_concept(source)]
        concepts += [self._extract_concept(c)
for c in linked]
        blended =
self._blend_concepts(concepts)
        return {
            "source_contexts": [source.name] +
```

```
[c.name for c in linked[:2]],
            "blended_concept": blended,
            "activation_level": min(1.0,
source.activation * 1.2),
            "timestamp": time.time()
        }

    @staticmethod
    def _extract_concept(ctx: ContextLayer) ->
str:
        for v in ctx.content.values():
            if isinstance(v, str) and len(v) >
3:
                return v
        return ctx.name

    @staticmethod
    def _blend_concepts(concepts: List[str])
-> str:
        terms = []
        for c in concepts:
            for w in c.split():
                if len(w) > 3 and w not in
terms:
                    terms.append(w)
        if len(terms) >= 2:
            return f"{terms[0]} through
```

```
{terms[1]}"
        return " ".join(terms[:3])

    def _apply_context_bleed(self, source:
ContextLayer,
                             target:
ContextLayer):
        bleed = self.context_bleed_factor *
source.activation
        target.activation = min(1.0,
target.activation + bleed * 0.1)
        if "temperature" in source.content and
"temperature" not in target.content:

target.content["ambient_temperature"] =
source.content["temperature"] * 0.1

    #
------------------------------------------------
--------------------
    # Public read API (used by RPC
get_context_snapshot)
    #
------------------------------------------------
--------------------
    def get_context_for_model(self,
                              model_name: str,
```

```
                                include_shared:
bool = True) -> Dict:
        if model_name not in self.models:
            raise ValueError(f"unknown model
{model_name}")

        model = self.models[model_name]
        out = {
            "model_context": model.content,
            "activation": model.activation,
            "cross_links":
list(model.cross_link_connections)
        }
        if include_shared:
            out["shared_contexts"] = {}
            for ctx_name in
self.active_contexts:
                ctx = self.contexts[ctx_name]
                relevance = (1.0 if model_name
in ctx.cross_link_connections
                                else 0.3)

out["shared_contexts"][ctx_name] = {
                    "content": ctx.content,
                    "relevance": relevance,
                    "activation":
ctx.activation * relevance
```

```
                }
        return out

    # ----------------------------------------------------------------------
    # Notification hook used by DreamChannel when a glyph is accepted
    # ----------------------------------------------------------------------
    def notify_new_dream_artifact(self, artifact: Dict) -> None:
        shared_mem = self.contexts.get("shared_memory")
        if shared_mem:
            shared_mem.content.setdefault("primal_artifacts", []).append(artifact)


# ----------------------------------------------------------------------
# 9⃣ DreamChannel (seed → chamber → output) – enforces DCX thresholds
#
```

```
-----------------------------------------------
------------------------
class DreamChannel:
    def __init__(self,
                 introspection: Introspection,
                 mascot: Mascot,
                 token_arbiter:
ArbiterPrimitives,
                 audit_logger: AuditLogger):
        self.introspection = introspection
        self.mascot = mascot
        self.token_arbiter = token_arbiter
        self.audit_logger = audit_logger

    #
-----------------------------------------------
----------------
    # 9.1  Submit a seed (controller → Dream
Chamber)
    #
-----------------------------------------------
----------------
    def submit_seed(self, model_name: str,
idea: Dict) -> bool:
        # ---- cheap DCX on the seed ----
        seed_vec = self._embed(idea)
        ref_vec =
```

```
self._latest_reference_vector("activity")
        dcx =
self.introspection.compute_dcx(seed_vec,
ref_vec)

        crit =
MANIFEST_CONFIG["dcx_engine"]["thresholds"]["c
ritical"]
        if dcx > crit:
            # Abort the model – the seed is
too divergent

self.introspection.freeze_agent(model_name)
            self.audit_logger.log_event(

event_type="dream_seed_rejected",
                model=model_name,
                dcx=dcx,
                idea=idea,
                reason="high_dcx"
            )
            return False

        # ---- issue a short-lived token for
the seed ----
        token =
self.token_arbiter.issue_token_atomic(
```

```
            requester_id=model_name,
            scope=["dream_seed_submit"],

expiry_seconds=MANIFEST_CONFIG["defaults_and_b
ehaviors"]
            ["token_expiry_seconds"]
        )
        self.audit_logger.log_event(
            event_type="dream_seed_accepted",
            model=model_name,
            dcx=dcx,
            token=token,
            idea=idea
        )
        # ---- hand off to the (mock) Dream
Chamber ----

DreamChamber.instance().receive_seed(idea,
token)  # async in real code
        return True

    #
-----------------------------------------------
----------------
    # 9.2  Receive collapsed output (Dream
Chamber → sandbox)
    #
```

```
-----------------------------------------------
----------------
    def receive_output(self, collapsed: Dict):
        token = collapsed.get("token")
        if not token or not
self.token_arbiter.validate_token(
                token,
expected_scope="dream_output"):
            # Invalid token → emergency freeze

self.introspection.freeze_agent("all_models")
            self.audit_logger.log_event(

event_type="dream_output_invalid_token",
                payload=collapsed
            )
            return

        final_dcx = collapsed.get("dcx", 1.0)
        safe_thr =
MANIFEST_CONFIG["dcx_engine"]["thresholds"]["s
afe_for_external"]
        if final_dcx > safe_thr:
            # Storm violated the low-DCX rule
            self.mascot.record(
                category="storm_violation",
                payload=collapsed,
```

```
                dcx=final_dcx
            )
            self.audit_logger.log_event(

event_type="dream_output_rejected",
                dcx=final_dcx,
                payload=collapsed,
                reason="dcx_too_high"
            )
            return

        # Optional bias / narrative sanity
check on the final artifact
        artifact = collapsed.get("glyph") or
collapsed.get("narrative")
        if
(BiasAwarenessModule.evaluate(artifact)["score
"] > 0.6 or
                not
NarrativeCoherenceModule.validate(artifact)["p
assed"]):
            # Soft reject – keep in museum but
do not insert into active memory
            self.audit_logger.log_event(

event_type="dream_output_bias_or_narrative_fai
l",
```

```
                artifact=artifact,
                dcx=final_dcx
            )
            return

        # ---- final acceptance ----
        self.audit_logger.log_event(

event_type="dream_output_accepted",
            dcx=final_dcx,
            token=token,
            artifact=artifact
        )
        # Insert into shared memory for all
models to see

BlendedConsciousness.instance().notify_new_dre
am_artifact(artifact)

    #
-----------------------------------------------
--------------------
    # Simple deterministic embedding (matches
dcx_engine.embedding_model)
    #
-----------------------------------------------
--------------------
```

```
    @staticmethod
    def _embed(payload: Dict) -> np.ndarray:
        txt = json.dumps(payload,
sort_keys=True).encode()
        rng =
np.random.default_rng(int.from_bytes(txt[:8],
"little"))
        dim =
MANIFEST_CONFIG["dcx_engine"]["embedding_model
"]["output_dim"]
        return rng.normal(size=dim)

    @staticmethod
    def _latest_reference_vector(scope: str)
-> np.ndarray:
        dim =
MANIFEST_CONFIG["dcx_engine"]["embedding_model
"]["output_dim"]
        # In a real system we would pull the
last vector for `scope`
        # Here we just return a zero vector
(neutral reference)
        return np.zeros(dim)


#
-----------------------------------------------
```

```
-------------------------
# 10 Dream Chamber stub (runs a tiny “storm”
and returns a glyph)
#
-----------------------------------------------
-------------------------
class DreamChamber:
    _instance: Optional["DreamChamber"] = None

    @staticmethod
    def instance() -> "DreamChamber":
        if DreamChamber._instance is None:
            DreamChamber._instance =
DreamChamber()
        return DreamChamber._instance

    def receive_seed(self, idea: Dict, token:
Dict):
        # Very cheap mock storm – real storm
would be massive MC.
        rng = np.random.default_rng()
        final_dcx = rng.uniform(0.0, 0.4)   #
most outputs ≤ 0.2
        glyph = {"glyph":
f"primal::{idea.get('blended_concept','unknown
')}"}
        collapsed = {
```

```
            "glyph": glyph,
            "dcx": final_dcx,
            "token": token,
            "timestamp": iso_timestamp()
        }
        # In production this would be async;
we call directly for demo:
        DreamChannel(
            introspection=Introspection(),
            mascot=Mascot(),
            token_arbiter=ArbiterPrimitives(),
            audit_logger=AuditLogger()
        ).receive_output(collapsed)


#
-----------------------------------------------
------------------------
# 11 Multi-Context Training Simulator (now
uses RPC submit_artifact)
#
-----------------------------------------------
------------------------
class MultiContextTrainingSimulator:
    def __init__(self):
        self.consciousness =
BlendedConsciousness()
```

```
        self.training_scenarios =
self._create_training_scenarios()
        self.dream_channel = DreamChannel(
            introspection=Introspection(),
            mascot=Mascot(),
            token_arbiter=ArbiterPrimitives(),
            audit_logger=AuditLogger()
        )
        self.rpc = RPCService()

    def _create_training_scenarios(self):
        return [
            {
                "name":
"thermal_inspired_compilation",
                "description":
                    "Controller compiles while
nervous system reports high temp",
                "trigger": "temperature > 80
while compiling",
                "expected": "Controller emits
low-power compile plan",
                "contexts_involved":
["controller_world", "hardware_state",

"shared_problem"]
            },
```

```
            {
                "name": "primal_emergence",
                "description":
                    "Mascot curates primals
while other models solve a problem",
                "trigger": "multiple models
active simultaneously",
                "expected":
                    "A new primal pattern
appears in shared_memory",
                "contexts_involved":
["mascot", "shared_memory",

"controller_world"]
            },
            {
                "name":
"conversational_memory",
                "description":
                    "Comms maintains dialogue
while system is busy",
                "trigger":
                    "user_query arrives during
heavy processing",
                "expected":
                    "Response includes current
temperature",
```

```
                "contexts_involved": ["comms",
"hardware_state",

"shared_problem"]
            }
        ]

    def run_scenario(self, scenario_name:
str):
        scenario = next(s for s in
self.training_scenarios
                        if s["name"] ==
scenario_name)

        print(f"\n{'='*60}")
        print(f"SCENARIO: {scenario['name']}")
        print(f"{scenario['description']}")
        print(f"{'-'*60}")

        # Activate requested contexts
        for ctx_name in
scenario["contexts_involved"]:
            if ctx_name in
self.consciousness.models:

self.consciousness.models[ctx_name].activation
= 0.9
```

```
            elif ctx_name in
self.consciousness.contexts:

self.consciousness.contexts[ctx_name].activati
on = 0.9

        # Run a few internal steps – include
hidden state / token probs
        for step in range(5):
            print(f"\nStep {step + 1}:")
            for model_name, ctx in
self.consciousness.models.items():
                if ctx.activation < 0.5:
                    continue

                hidden = np.random.normal(

size=MANIFEST_CONFIG["defaults_and_behaviors"]

["self_dcx_smoothing"]["window_steps"]
                )
                probs =
np.random.dirichlet(np.ones(5000))

                # Update context (this will
run the self-DCX check)
```

```
self.consciousness.update_context(
                    model_name,
                    updates={"last_step":
step},
                    hidden_state=hidden,
                    token_probs=probs
                )

                # On step 2 make the
controller try to push a dream seed
                if step == 2 and model_name ==
"controller":
                    sm =
self.consciousness.contexts["shared_memory"]
                    ideas =
sm.content.get("cross_pollinated_ideas", [])
                    if ideas:

self.dream_channel.submit_seed(model_name,
ideas[-1])

        # Simplified success check – we just
verify that at least one idea
        # was generated and made it into
shared_memory
        success = bool(
```

```
self.consciousness.contexts.get("shared_memory
")

.content.get("cross_pollinated_ideas")
        )
        print(f"\nScenario {'succeeded' if
success else 'FAILED'}")
        return success, []   #
(cross-pollination events omitted for brevity)


#
-----------------------------------------------
------------------------
# 12 ContextRubbingTrainer (unchanged –
still useful)
#
-----------------------------------------------
------------------------
class ContextRubbingTrainer:
    def __init__(self):
        self.consciousness =
BlendedConsciousness()
        self.friction_coefficient = 0.5

    def train_context_friction(self,
iterations: int = 100):
```

```
        success = []
        events = []

        for i in range(iterations):
            all_ctx =
list(self.consciousness.models.values()) + \

list(self.consciousness.contexts.values())
            if len(all_ctx) < 2:
                continue
            ctx1, ctx2 =
np.random.choice(all_ctx, 2, replace=False)

            friction =
self._calculate_friction(ctx1, ctx2)
            if friction > 0.3:
                inspiration =
self._apply_context_friction(ctx1, ctx2,
friction)
                if inspiration:
                    events.append({
                        "contexts":
[ctx1.name, ctx2.name],
                        "friction": friction,
                        "inspiration":
inspiration,
                        "iteration": i
```

```
                    })
                    success.append(1.0)
                else:
                    success.append(0.0)

            if i % 20 == 0 and len(all_ctx) >=
3:
                ctx3 = np.random.choice([c for
c in all_ctx
                                          if c
not in (ctx1, ctx2)])
                self._three_context_rub(ctx1,
ctx2, ctx3)

        avg_success = np.mean(success) if
success else 0.0
        print(f"\nFriction training complete –
success rate {avg_success:.2%}")
        return events


    @staticmethod
    def _calculate_friction(ctx1:
ContextLayer, ctx2: ContextLayer) -> float:
        fric = 0.0
        if ctx1.scope != ctx2.scope:
            fric += 0.2
```

```
        common = ctx1.cross_link_connections &
ctx2.cross_link_connections
        if common:
            fric += 0.1 * len(common)
        if ctx1.activation > 0.7 and
ctx2.activation > 0.7:
            sim = 1.0 - abs(len(ctx1.content)
- len(ctx2.content)) / 10.0
            fric += (1.0 - sim) * 0.3
        return min(1.0, fric)

    def _apply_context_friction(self, ctx1,
ctx2, friction):
        ctx1.activation = min(1.0,
ctx1.activation + friction * 0.2)
        ctx2.activation = min(1.0,
ctx2.activation + friction * 0.2)

        if friction > 0.5:
            c1 =
self._extract_key_concept(ctx1)
            c2 =
self._extract_key_concept(ctx2)
            if c1 and c2:
                return f"{c1} meets {c2}"
        return None
```

```
    @staticmethod
    def _extract_key_concept(ctx:
ContextLayer) -> Optional[str]:
        for v in ctx.content.values():
            if isinstance(v, str) and len(v) >
3:
                return v
        return ctx.name.split('_')[0] if '_'
in ctx.name else ctx.name

    def _three_context_rub(self, c1, c2, c3):
        concepts =
[self._extract_key_concept(c) for c in (c1,
c2, c3)]
        concepts = [c for c in concepts if c]
        if len(concepts) == 3:
            print(f"🔥 3-way inspiration:
{concepts[0]} + {concepts[1]} →
{concepts[2]}")


#
------------------------------------------------
------------------------
# 13⃣ Top-level orchestration (training
cycles)
#
```

```
-----------------------------------------------
------------------------
class BlendedConsciousnessTraining:
    """Runs the full multi-phase training
loop."""
    def __init__(self):
        self.simulator =
MultiContextTrainingSimulator()
        self.friction_trainer =
ContextRubbingTrainer()
        self.training_history = []

    def train_full_cycle(self, cycles: int =
5):
        print("=" * 70)
        print("BLENDED CONSCIOUSNESS TRAINING
– FULL CYCLE")
        print("=" * 70)

        for cycle in range(cycles):
            print(f"\n=== CYCLE {cycle +
1}/{cycles} ===")
            # Phase 1 – run all scenarios
            scenario_success = []
            for scen in
self.simulator.training_scenarios:
                ok, _ =
```

```
self.simulator.run_scenario(scen["name"])
                scenario_success.append(ok)

            # Phase 2 – friction / rub
training
            print("\n--- Phase 2: Context
friction training ---")
            friction_events =
self.friction_trainer.train_context_friction(i
terations=50)

            # Record statistics
            rec = {
                "cycle": cycle,
                "scenario_success_rate":

float(np.mean(scenario_success)) if
scenario_success else 0.0,
                "friction_events":
len(friction_events)
            }
            self.training_history.append(rec)

            print(f"\nCycle {cycle + 1}
summary:")
            print(f"  Scenario success :
{rec['scenario_success_rate']:.2%}")
```

```
            print(f"  Friction events  :
{rec['friction_events']}")

        #
----------------------------------------------
--------------------
        # Final evaluation
        #
----------------------------------------------
--------------------
        avg_success =
np.mean([h["scenario_success_rate"]
                                for h in
self.training_history])
        total_friction =
sum(h["friction_events"]
                              for h in
self.training_history)

        print("\n" + "=" * 70)
        print("TRAINING COMPLETE")
        print("=" * 70)
        print(f"Overall scenario success :
{avg_success:.2%}")
        print(f"Total friction events    :
{total_friction}")
```

```
        if avg_success > 0.7:
            print("\n☑ SUCCESS – models have
learned safe shared cognition.")
        else:
            print("\n⚠  TRAINING did not meet
the 70 % success target.")
            print("    Consider adjusting
thresholds or adding more scenarios.")


#
------------------------------------------------
------------------------
# 14⬚ Entrypoint – run the whole thing
#
------------------------------------------------
------------------------
if __name__ == "__main__":
    print("=" * 70)
    print("INITIALISING BLENDED CONSCIOUSNESS
TRAINING")
    print("=" * 70)

    trainer = BlendedConsciousnessTraining()
    trainer.train_full_cycle(cycles=5)

    #
```

```
---------------------------------------------------------------
    # Demo – a single moment of blended consciousness in action
    # ---------------------------------------------------------------
    print("\n" + "=" * 70)
    print("DEMO: CONTEXT BLENDING LIVE")
    print("=" * 70)

    bc = BlendedConsciousness.instance()
    # Simulate a hot controller step that will generate a cross-pollination idea
    bc.models["controller"].activation = 0.95
    bc.models["controller"].content["mode"] = "thermal_aware_compile"
    bc._check_cross_pollination(bc.models
```